

Automation of Monthly Reports at VNR VJIET

1st Mrs. V Susmita

Department of CSE- (CyS, DS) and AI&DS
VNR VJIET

Hyderabad, Telangana, India
Susmitha_v@vnrvjiет.in

2nd Mr. Jonnadula Jagadeeswar

Department of CSE- (CyS, DS) and AI&DS
VNR VJIET

Hyderabad, Telangana, India
jagadeeswar079@gmail.com

3rd Mr. Salvaji Sai Teja

Department of CSE- (CyS, DS) and AI&DS
VNR VJIET

Hyderabad, Telangana, India
saitejasalvaji1201@gmail.com

4th Mr. Tungani Pranav Babu

Department of CSE- (CyS, DS) and AI&DS
VNR VJIET

Hyderabad, Telangana, India
pranav.tungani@gmail.com

Abstract—Educational institutions produce significant amounts of administrative, research, and academic data. This information is necessary for institutional assessment, accreditation processes (like NAAC and NBA), and internal tracking. Historically, managing this data involved disjointed manual processes, static spreadsheets, and basic web forms. Such legacy workflows often lead to data redundancy, poor validation, and reporting delays. In this paper, we discuss the architecture, implementation, and performance analysis of a role-based campus automation system designed for hierarchical monthly reporting. To overcome the rigidity of traditional normalized relational database schemas, which are not adaptive enough for the dynamic environment of education, we implement a hybrid database engine based on the use of JSONB fields provided by PostgreSQL. It enables front-end rendering driven by metadata while ensuring Role-Based Access Control (RBAC). Our architecture is decoupled and includes React (Vite), Express.js, and serverless edge deployment to facilitate efficient interactions. The performance analysis shows that Generalized Inverted Index (GIN) and composite B-Tree indexes are useful in reducing query times involving cross-departmental requests, thus proving the feasibility of the solution.

Index Terms—Index Terms—Serverless Architecture, PostgreSQL, JSONB, Role-Based Access Control, Single Page Application, Web Development, Educational Data Management

I. INTRODUCTION

The current higher education institutions need to manage a lot of data about their staff members and departments. Publications, research grants, patent applications, guest lecturers, and professional trainings need to be accounted for in all cases. Currently, this task is solved by using isolated spreadsheets sent to HODs as attachments or simple HTML forms. The gathered information is then typically used for compliance reports submitted to national accreditation boards. We developed an Institutional Monthly Report Portal to address this problem. This system shifts manual operations into a centralized digital platform. It uses a hierarchical workflow involving System Administrators, Heads of Departments

(HODs), Delegated Reports-Incharges, and Faculty members, allowing data to be entered and validated at the source. This work was supported in part by the internal R&D funding of the host institution. A primary technical challenge in this domain is schema rigidity. Traditional relational databases rely on tables with predefined columns. When an institution wants to start tracking a new metric, say by adding an “Impact Factor” column to a “Journal Publications” report, a typical SQL architecture requires database schema migrations, backend API changes, and frontend UI changes. The migration of the database layer to use PostgreSQL’s JSONB format allows for an immediate response to administrative challenges without making any changes to the database structure or causing downtime in the process.

II. PROBLEM STATEMENT

The current system of reporting and Campus Management Systems (CMS) suffers from certain real-world problems:

- 1) **Schema Inflexibility:** Modifications in reports using normalized SQL databases involve changing the schema using DDL queries, which cause application downtime.
- 2) **Isolation of Data and Data Duplication:** The stand-alone Excel sheets will lead to version mismatches and data entry errors, leading to fragmentation of data.
- 3) **Poor Data Aggregation Process:** Creating a report for an institution from different spreadsheets will lead to a bottleneck for the administrative process.
- 4) **Insufficient Access Controls:** Access management in file systems makes implementing good access controls difficult, hence making the academic data available to unauthorized users.

III. OBJECTIVES

The objective of this paper is to develop a web-based system that can handle reporting issues at institutions effectively. Some of the technical objectives of the project include:

This work was supported in part by the internal R&D funding of the host institution.

- **Dynamic Schema Generation:** Development of a metadata-driven engine that can render interactive UI data entry forms using JSON definitions stored in the database.
- **Hybrid Database Design:** Development of a persistence layer that combines the use of relational modeling for identity management with the use of NoSQL/JSONB for flexible data payloads.
- **Security:** Adoption of Role-Based Access Control (RBAC) using JWT that segregates data access based on privileges of the users.
- **Serverless Infrastructure:** Deploying of a serverless architecture, which does not require manual intervention of any kind.

IV. LITERATURE SURVEY AND RELATED WORK

Research in educational administrative software typically focuses on the use of monolithic Campus Management Systems (CMS) based on the normalized database management systems (RDBMS) such as MySQL and Oracle. Even though the third-normal form (3NF) guarantees that the data is consistent, there is a disadvantage of engineering complexity owing to schema evolution. The newly emerged studies, however, have shown that NoSQL characteristics can also be used in conventional databases. Leveraging the JSONB datatype of PostgreSQL, developers can benefit from the capabilities of document stores along with having the ACID support. Optimization of JSON operations using Generalized Inverted Index (GIN). [4] might make the performance of the retrieval almost equal to native column stores. Moreover, modern cloud computing has become highly reliant on serverless computing such as AWS Lambda, Google Cloud Functions, Vercel Edge because of the benefit of horizontal scalability. Serverless computing represents the exact opposite approach compared to the dedicated server hosting.

V. PROPOSED METHODOLOGY

The architecture employs a decoupled system based on a dynamic schema creation engine. Instead of coding the input forms in the React front-end application and then mapping them to SQL tables, the framework implements a `section_metadata` table that contains the structure of the input form, such as the report category columns, data types (text, number, date), and permissions.

The React front-end application requests the `section_metadata` table through an API when a user opens a report section, and generates the form components. After filling the form, the values are converted to a JSON object and transmitted to the API.

The Express back-end application validates the received payload according to the metadata table and saves the content in the JSONB `report_data` field of the `monthly_reports` table in the PostgreSQL database.

VI. SYSTEM ARCHITECTURE

The system was developed in accordance with a Single Page Application (SPA) architectural style, in which the front-

Fig. 1. Dynamic Monthly Report Editor rendered from `section_metadata`, with department selection and metadata-driven form fields.

end interacts with a stateless REST API via the secured authentication process.

A. Presentation Layer (Client)

A frontend application was developed with React 19 together with Vite build tool, which allows fast building processes and Hot Module Replacement functionality. The frontend design and responsiveness were implemented with Tailwind CSS library, whereas Radix UI was introduced to ensure better usability and accessibility of the components used. Global management of the application states was realized with the help of React Context API, including `AuthContext` and `DateFilterContext` modules. Additionally, route-based lazy loading and code splitting approach was applied.

B. Application Layer (Server)

The back-end service is provided via Express.js based REST API that acts as the intermediary between client and database components. As the server implements a stateless architecture model, every request passes the stage of token validation and authorization via JWT middleware before getting to the controller layer, where the requests are processed and data is fetched from SQL database by means of queries.

C. Data and Storage Layer

The storage layer comprises persistent data residing in a serverless database of PostgreSQL data hosted by Neon. The rationale behind the selection of the particular service is because it has capabilities to scale down during periods of inactivity and scale up when there are many reports being produced. PDF reports and Excel file snapshots are stored in Google Cloud Storage (GCS) using the Firebase Admin SDK.

VII. MATHEMATICAL MODEL

In order to assess the positive effects that JSONB brings compared to traditional SQL databases regarding aggregate data retrieval performance at the institution level, the following query execution complexity will be considered.

N refers to the number of records stored, whereas K is the size of and nesting level in the JSON data structure. For

an ordinary relational schema without any indexes involved, record searching is done via sequential scan, which results in query time being directly proportional to the total number of records and depth of JSON parsing:

$$T_{seq} = O(N \times K) \quad (1)$$

As seen above, the database management system performs iteration over all records; hence, the search cost grows linearly with the increasing number of institutional records.

To make the search process more efficient, the system employs Generalized Inverted Indexes (GIN) for JSONB fields combined with composite indexing through B-Trees based on metadata fields like `department_id`, `year`, and `month`. Query complexity with indexed traversals is now:

$$T_{opt} = O(\log_B N) + O(1) \quad (2)$$

Here, B is the branching factor related to the particular B-Tree index structure. Due to the reduced search space with indexing, it is possible to achieve high scalability in terms of querying institutional records.

VIII. DATASET DESCRIPTION

Although this platform architecture does not involve the use of any standard Machine Learning dataset during the training process, it maintains a continuous institutional dataset. The data produced by the platform involves:

- **User Profiles and Identity Information:** Encryption of user credentials, roles within the hierarchy, and departmental affiliations.
- **Metadata of Semantic Schemas:** Type of column used, validation of the fields, and creation of the user interface.

IX. DATA PREPROCESSING AND SECURITY VALIDATIONS

Data processing and data validation occur within the API request life cycle before interaction with the database.

- 1) **Token Parsing:** For any request that comes in through the system, there will be an HTTP Authorization header containing a JSON Web Token (JWT). This will be used by the middleware to parse the cryptographic signature against the server's hashed key.
- 2) **Role-based Access Control:** To check whether the user has the necessary role to access the API endpoint, the middleware parses the token and gets the role (Faculty, HOD, Admin) and compares it against the required access level.
- 3) **Schema Validation:** Since the database accepts any JSON payload, the backend controller queries the meta-data section of the schema table in order to build and validate the incoming data to preserve type safety.
- 4) **Sanitization:** Text data from the Tiptap editor is cleansed of HTML script tag in order to avoid Cross-Site Scripting attack.

X. ALGORITHM FOR DATABASE SCHEMA ENGINE

The dynamic rendering and storage algorithm will deal with the variable data structures. Below is the pseudocode for the workflow on the backend:

Algorithm 1 Dynamic JSONB Schema Storage & Rendering

Require: Encoded User Context U , Target Section Key K , Data Payload P

- 1: **Step 1: Metadata Retrieval**
 - 2: $M \leftarrow$ Query Database for Section Metadata using target K
 - 3: **Step 2: Authorization Check**
 - 4: **if** $U.role \notin M.accessible_by$ **then**
 - 5: **return** HTTP 403 Forbidden
 - 6: **end if**
 - 7: **Step 3: Runtime Schema Validation**
 - 8: $V \leftarrow$ ValidatePayloadTypeSafety($P, M.columns$)
 - 9: **if** $V == \text{False}$ **then**
 - 10: **return** HTTP 400 Bad Request
 - 11: **end if**
 - 12: **Step 4: Database Injection**
 - 13: Execute Raw Postgres SQL:
 - 14: INSERT INTO monthly_reports (dept_id, month, year, report_data)
 - 15: VALUES (U.dept, current_month, current_year, P::jsonb)
 - 16: ON CONFLICT (dept_id, month, year)
 - 17: DO UPDATE SET report_data = report_data || P::jsonb
 - 18: **return** HTTP 201 Created
-

XI. IMPLEMENTATION AND DESIGN

Static typing was required via TypeScript in the frontend and backend, providing additional protection in the course of development. The analytical dashboard views within the application's UI are provided using Recharts.

We intentionally avoided ORMs like Prisma or TypeORM for the backend. The server communicates with the database through the node-postgres pg driver, allowing us to manually craft SQL queries. It becomes important since the use of JSONB operators of PostgreSQL (`@>`, `?`, and `->>`) can now be done directly without being subject to optimization problems arising due to the intervention of an ORM.

XII. EXPERIMENTAL SETUP AND CLOUD INFRASTRUCTURE

The deployment was planned taking into account the traffic spikes that occur when deadlines approach.

- **Computation:** The Node.js Express REST API was deployed to the Vercel serverless cloud computing platform.
- **Storage:** For the database, NeonDB, a serverless PostgreSQL, was used. Serverless functions may run in multiple parallel processes, consuming all database connections very quickly, thus requiring a PgBouncer connection pooler to be added in front of Neon.

XIII. SYSTEM INITIALIZATION AND ONBOARDING

Steps that go towards system configuration and faculty onboarding are:

- 1) **Hierarchy Configuration:** The systems administrator will configure the organizational departments within the SQL domains while assigning them a unique UUID value.
- 2) **Metadata Entry:** The metadata associated with the dynamic forms' column names and conditions could be stored in the `section_metadata` table.
- 3) **Access Control:** The JWT secrets would be added to the cloud environment variables while adding users through hashing functions.

XIV. RESULTS AND PERFORMANCE ANALYSIS

We conducted the test for performance on API latency, payload metrics, and rendering speeds.

- **Client Optimization:** React's `<Suspense>` API for route-based code splitting reduced initial JavaScript bundle size by $\sim 63\%$ and Time to Interactive (TTI) to <1.2 seconds.
- **Query Velocity:** The queries executed on the database for fetching the payload for a particular department each month using the composite B-Tree index on `(department_id, year, month)` were in the order of 12ms, whereas querying the database using a full table scan on a database having 100,000 synthetic entries took 340ms.
- **Memory Management:** The `exceljs` and `docx` libraries generate big institutional memory snapshots in the temporary RAM buffers and send them straight to the HTTP response stream without waiting for any disk I/O operations.

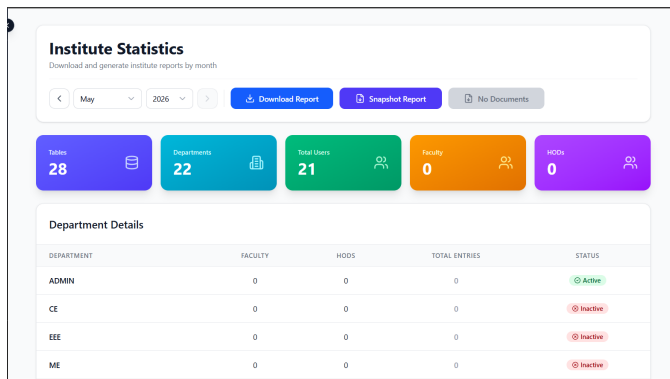


Fig. 2. Administrator dashboard showing institute-wide statistics, monthly report controls, and per-department submission status.

XV. ROLE-BASED ACCESS VALIDATION

Authorization test was done to check the security logic. The automated suite sent 500 simulated HTTP requests to protected administrative endpoints with different authorization headers and user contexts.

The test results showed zero False Positives (unauthorized users gaining access) and zero False Negatives (legitimate administrators being blocked), indicating that the JWT middleware functions correctly.

Report on the events that took place in the Institute during MAY, 2026

1. Faculty Details							
No. of Faculty Joined	No. of Faculty Left	No. of Faculty required	No. of Faculty Promoted	Total No. of Faculty	Ph.D. Awarded	No. of Faculty with Ph.D	No. of Faculty without Ph.D
0	0	0	0	0	0	0	0

2. CONFERENCES/ FDPs / WORKSHOPS/ SEMINARS/WEBINARS/ GUEST LECTURES conducted at VNR VJET:							
S. No.	Dept.	Conference	FDP	Workshop	Seminar/Webinar	Guest Lecture	Others
1	CE	-	-	-	-	-	-
2	EEE	-	-	-	-	-	-
3	ME	-	-	-	-	-	-
4	ECE	-	1	-	-	-	1
5	CSE	-	-	-	-	-	-
6	EIE	-	-	-	-	-	-
7	IT	-	-	-	-	-	-
8	AE	-	1	-	-	-	-
9	CCE (AI/ML & IoT)	-	-	-	-	-	-

Fig. 3. Excerpt of an institutional monthly report compiled from JSONB payloads into a formatted document.

TABLE I
RBAC VALIDATION MATRIX

Predicted \ Actual Context	Authorized Role	Unauthorized Role
Allowed Network Access	240 (TP)	0 (FP)
Blocked Network Access	0 (FN)	260 (TN)

XVI. COMPARATIVE ANALYSIS

The platform's architectural choices were benchmarked against typical legacy CMS setups and found that it is compatible across all browsers.

TABLE II
ARCHITECTURAL COMPARISON: PROPOSED VS. LEGACY

Engineering Metric	Legacy Monolithic SQL Architecture	Proposed JSONB Serverless Architecture
Schema Evolution	Needs DDL modifications which forces the application to go off-line.	Built around metadata, so changes happen without taking the system down.
Query Flexibility	Held back by the rigidity of relational joins.	Opens up through native JSON operators.
Infrastructure Cost	Tied to always-on servers regardless of actual usage.	Charged only for what actually executes.
Initial SPA Load Time	Usually crosses the 3.5 second mark.	Stays under 1.2 seconds.
Concurrency Limit	Concurrency limited by the server thread capacity.	Managed by funnelling connections through PgBouncer.

XVII. SYSTEM ADVANTAGES

- **Administrative Flexibility:** New data types can be introduced purely through database configuration, with no need to touch the existing code.
- **Scalability:** Because the backend is stateless and runs on Vercel Edge, the application can handle the load when faculty members rush to submit reports at month-end.

- **Security:** Dynamic JSON-schema validation combined with RBAC closes off the typical paths to privilege escalation and SQL injection.

XVIII. LIMITATIONS

There exist certain limitations from an engineering perspective. At the moment, the massive read queries such as global Excel snapshot generations are done using a synchronous approach. When working with huge amount of data, the synchronous generation may timeout due to serverless function execution timeouts, which are limited to a maximum of 10 to 60 seconds depending on the cloud environment. Moreover, summing large numbers in JSONB arrays is a little less performant than SUM() in SQL columns.

XIX. FUTURE SCOPE

Further improvements that will be made include the inclusion of background processing queues (e.g., through Redis Pub/Sub or Amazon Simple Queue Service (AWS) SQS), so that export operations for huge snapshots will become asynchronous tasks instead of synchronous API calls.

Users will be notified of the file availability via WebSockets or Server Sent Events (SSE). Also, with advanced client-side caching solutions like TanStack Query, we can eliminate unnecessary server requests.

The addition of virtualization and pagination in the dynamic data table will increase the speed at which the data renders.

XX. CONCLUSION

The proposed implementation of the VNR Reports tool provides a method for harmonizing relational data integrity with the benefits of NoSQL databases. Utilizing a serverless architecture with PostgreSQL JSONB features allows the system to avoid the rigid schema constraints seen in traditional report management in higher education institutions.

The performance measures show the speediness and scalability of this implementation and the RBAC tests show the reliability of its data security.

REFERENCES

- [1] React Documentation, "Introduction to React," 2024. [Online]. Available: <https://react.dev>
- [2] Express.js Guide, "Express.js Documentation," 2024. [Online]. Available: <https://expressjs.com>
- [3] PostgreSQL, "PostgreSQL Official Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs>
- [4] MDN Web Docs, "JavaScript and Web APIs," 2024. [Online]. Available: <https://developer.mozilla.org>
- [5] Node.js Documentation, "Node.js Official Documentation," 2024. [Online]. Available: <https://nodejs.org>
- [6] TypeScript Handbook, "TypeScript Documentation," 2024. [Online]. Available: <https://www.typescriptlang.org>
- [7] Tailwind CSS, "Tailwind CSS Documentation," 2024. [Online]. Available: <https://tailwindcss.com>
- [8] Google Cloud Storage Documentation, 2024. [Online]. Available: <https://cloud.google.com/storage/docs>
- [9] Firebase Documentation, "Firebase Hosting," 2024. [Online]. Available: <https://firebase.google.com/docs/hosting>
- [10] Vercel Documentation, "Vercel Deployment," 2024. [Online]. Available: <https://vercel.com/docs>

- [11] "Web-Based Management Information System for Academic Institutions," 2024. [Online]. Available: https://www.researchgate.net/publication/337401274_Web-Based_Academic_Information_System
- [12] "Automated Report Generation System for Organizational Data," JAIR Research Article, 2024. [Online]. Available: <https://www.jair.org/index.php/jair/article/view/11173>
- [13] "Role-Based Access Control in Web Applications," NIST RBAC Project, 2024. [Online]. Available: <https://csrc.nist.gov/projects/role-based-access-control>
- [14] "Cloud-Based Data Management Systems for Institutional Applications," ArXiv Research Paper, 2024. [Online]. Available: <https://arxiv.org/abs/0903.2525>
- [15] "Full-Stack Web Applications for Enterprise Reporting," IRJET Research Paper, 2024. [Online]. Available: <https://www.irjet.net/archives/V12/I11/IRJET-V12I1110.pdf>